

Production-Ready Deployment SOP

Project: Tyger Talent Analytics (TTA) — Production Stack (FastAPI, Gunicorn, Nginx, MariaDB, React)

Version: 1.0

Generated: 23 February 2026

Prepared by: DevOps / Engineering

Table of Contents

1	1. Purpose & Scope
2	2. Key Definitions and Roles
3	3. Environment Architecture & Diagram
4	4. Prerequisites & Access
5	5. Git Workflow & CI/CD (Automated)
6	6. Deployment Strategy (Automated + Manual Emergency)
7	7. Database Migration & Data Management
8	8. Backup, Restore & Disaster Recovery
9	9. Rollback Procedures & Release Management
10	10. Monitoring, Alerts & Post-deployment Checks
11	11. Security & Configuration Management
12	12. Incident Response & Runbook
13	13. Maintenance & Patching
14	14. Appendices (config snippets, commands, templates)
15	Version History

1. Purpose & Scope

This SOP defines the production deployment process for the TTA project. It standardizes automated CI/CD deployments, manual emergency procedures, rollback, database migrations, backups, monitoring, and incident response. Scope: backend (FastAPI + Gunicorn), frontend (React static build), database (MariaDB), reverse proxy (Nginx), systemd services on Ubuntu servers.

2. Key Definitions and Roles

Product Owner	Defines release windows and approves major releases.
Tech Lead	Approves PRs affecting architecture, migrations; triages incidents.
Deploy Engineer (Authorized)	Triggers production deployments, monitors release.
DB Owner / Data Engineer	Approves schema changes, validates backups and migrations.
On-call Engineer	First responder for P0/P1 incidents.
Security Owner	Manages secrets and cert renewals.

3. Environment Architecture & Diagram

Environments:

- Development: Local developer environments. No production data. Frequent merges to feature branches.
- Staging: Mirrors production configuration. Used for integration testing and final QA.
- Production: Live environment handling real users and data.

Architecture Overview (flow):

User → DNS → Load Balancer (optional) → Nginx (SSL, static files) → Gunicorn (unix socket) → FastAPI → MariaDB

Key design notes:

- Use unix sockets for Gunicorn to Nginx proxy (e.g., /run/tta.sock) to limit public exposure.
- Keep environment variables and secrets out of Git; use secrets manager or encrypted .env on server.
- Staging must be as close as possible to production (same DB engine/version, similar config).

4. Prerequisites & Access

- Server prepared with Ubuntu LTS, security updates applied.
- User accounts: dedicated deploy user (no root SSH directly), sudo as needed.
- SSH keys: deploy engineer public key in /home/deploy/.ssh/authorized_keys.
- systemd unit file present at /etc/systemd/system/tta.service and tested.
- Nginx site config available in /etc/nginx/sites-available/tta and symlinked to sites-enabled.
- Backups configured and tested (mysqldump and file backups).
- Monitoring and alerting endpoints configured (Sentry/Prometheus/Grafana or similar).

5. Git Workflow & CI/CD (Automated)

Branching & Release Rules:

- Use protected 'main' branch: merges only via PR after 1+ approvals and passing CI.
- Feature branches: feature/. Bugfix branches: fix/. Hotfix branches: hotfix/.
- Tagging policy: semantic versioning vMAJOR.MINOR.PATCH (e.g., v1.2.3).
- Release notes: maintain CHANGELOG.md with short summary and migration notes.

CI/CD Pipeline (recommended):

- On PR: run linters, unit tests, static type checks (mypy), and build frontend unit tests.
- On merge to main: run full test suite, build frontend production artifact, run DB migration dry-run on staging, build backend package.
- On tag push (release): trigger production deploy job (see workflow snippet in Appendix).
- Ensure artifact immutability: build once, store artifact (tarball/docker image) and deploy same artifact to staging then production.

6. Deployment Strategy (Automated + Manual Emergency)

Preferred Deployment (Automated via CI/CD):

- CI builds artifact, uploads to secure artifact storage (S3 or equivalent) and SSHs to production deploy host.
- Deployment uses a release directory pattern: /srv/tta/releases/ with symlink /srv/tta/current to the active release for atomic switch and quick rollback.
- After artifact extraction, run migrations in a transaction-safe way and perform smoke tests before switching symlink.
- Use systemd to manage Unicorn and ensure graceful worker restart (systemctl restart tta).

Manual Emergency Deployment (when CI is unavailable):

- Only authorized deploy engineer performs manual deploy following checklist.
- Steps: pull tag, create release dir, install deps in virtualenv, run migrations, update symlink, restart systemd, verify logs and smoke tests.
- Document the manual deploy in deployment log with time, operator, git tag, and notes.

7. Database Migration & Data Management

Migration Principles:

- Migrations must be backward compatible where possible (avoid destructive operations in the same release).
- Use migration tool (Alembic or equivalent).
- Run migrations on staging and validate before production run.
- If destructive changes required, plan multi-step deployment: add new column → backfill → switch reads/writes → remove old column in later release.

Production Migration Steps:

- Create DB backup before running migrations: `mysqldump --single-transaction --quick --routines --triggers -u user -p dbname > backup.sql`
- Run migration in a transaction when supported.
- Validate post-migration schema and sample queries.
- If migration fails, rollback using DB restore and revert code to previous tag.

8. Backup, Restore & Disaster Recovery

Backup Strategy:

- Daily full logical DB backup (mysqldump) stored off-server (S3 or secure object store).
- Incremental binary backups if DB size requires (use mariabackup/Percona XtraBackup).
- Daily verification: automated test restore to a sandbox every 7 days.
- Application config backups (nginx, systemd unit files, .env) stored encrypted.

Disaster Recovery (DR) Targets:

- Recovery Time Objective (RTO): 1 hour for P0 (application down) with staffed response.
- Recovery Point Objective (RPO): <= 4 hours for critical data; shorter if business requires.
- DR Runbook location: /runbooks/dr.md (ensure access).

9. Rollback Procedures & Release Management

Atomic Release Pattern (recommended):

- Deploy to new release directory, perform migrations if safe, run smoke tests, then atomically switch symlink /srv/tta/current to new release.
- Rollback: switch symlink to previous release directory and restart service. If DB changes performed, follow DB rollback plan.
- Keep last N releases (e.g., 5) for quick rollbacks.

10. Monitoring, Alerts & Post-deployment Checks

Monitoring Components:

- Application errors: Sentry or equivalent for error capturing and stack traces.
- Metrics: Prometheus + Grafana for latency, request rates, error rates, DB connections, CPU, memory.
- Logs: Centralized logging via journalctl + filebeat + ELK or a managed log service.
- Synthetic checks: periodic health-check endpoints and uptime monitors (e.g., Pingdom, UptimeRobot).

Post Deployment Verification (30-60 minutes):

- Verify 200 responses on key endpoints and smoke tests.
- Check error rate does not spike above baseline.
- Monitor DB slow queries and locks.
- Confirm frontend assets load and CDN (if used) caches updated.

11. Security & Configuration Management

- Secrets Management: use Vault/SSM/Key Management Service to store DB credentials and SECRET_KEY; do not store plaintext .env in Git.
- Firewall: allow only required inbound ports (80, 443, SSH from admin IPs).
- SSH: disable password auth, use key-based auth, disable root login.
- OS Patching: apply security updates on a scheduled maintenance window and test on staging before production.
- Certificates: automate certbot renewals and monitor expiry.

12. Incident Response & Runbook

Severity Classification:

P0	Application down for users. Immediate page to on-call, start rollback if unresolved in 10 minutes.
P1	Core functionality broken for significant subset. Immediate triage, possible hotfix.
P2	Non-critical functionality broken; schedule fix in next patch.

Incident Runbook (high level):

- Detect: alert fires (Sentry/Prometheus) or user report.
- Triage: on-call confirms severity and notifies stakeholders (slack/phone).
- Mitigate: apply rollback or hotfix; run tests.
- Recover: validate restoration and monitor.
- Post-mortem: document timeline, root cause, action items.

13. Maintenance & Patching

- Schedule maintenance windows and notify users in advance for non-emergency patching.
- Test all patches on staging before applying to production.
- Reboot servers only when necessary; ensure service auto-start via systemd.
- Rotate credentials annually or on compromise.

14. Appendices (config snippets, commands, templates)

A. Example systemd service (tta.service):

```
[Unit] Description=tta unicorn daemon After=network.target [Service] User=deploy Group=www-data  
WorkingDirectory=/srv/tta/current EnvironmentFile=/srv/tta/current/.env  
ExecStart=/srv/tta/current/venv/bin/gunicorn -k uvicorn.workers.UvicornWorker --workers 4 --bind  
unix:/run/tta.sock tta.main:app Restart=on-failure [Install] WantedBy=multi-user.target
```

B. Example Nginx site config (snippet):

```
server { listen 80; server_name your.domain.com; return 301 https://$host$request_uri; } server { listen 443 ssl;  
server_name your.domain.com; ssl_certificate /etc/letsencrypt/live/your.domain.com/fullchain.pem;  
ssl_certificate_key /etc/letsencrypt/live/your.domain.com/privkey.pem; root /srv/tta/current/frontend_build; index  
index.html; location /api/ { proxy_pass http://unix:/run/tta.sock; proxy_set_header Host $host; proxy_set_header  
X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; } }
```

C. GitHub Actions (example) - simplified deploy job (concept):

```
name: CI/CD on: push: tags: ['v*.*.*'] jobs: build-and-deploy: runs-on: ubuntu-latest steps: - uses:  
actions/checkout@v3 - name: Set up Python uses: actions/setup-python@v4 with: python-version: '3.13' - name:  
Install deps and run tests run: | pip install -r backend/requirements.txt pytest -q - name: Build frontend run: | cd  
frontend && npm ci && npm run build - name: Package and upload artifact run: | tar -czf release.tar.gz backend  
frontend # upload to artifact store or S3 - name: SSH Deploy to Prod uses: appleboy/ssh-action@v0.1.7 with:  
host: ${ secrets.PROD_HOST } username: deploy key: ${ secrets.SSH_KEY } script: | set -e mkdir -p  
/srv/tta/releases/$GITHUB_RUN_ID # extract artifact, install venv, migrate, smoke-test, update symlink, restart  
systemd
```

D. Emergency manual deploy checklist (short):

- Confirm authorized operator and deployment window.
- Take DB backup: mysqldump ... > backup.sql
- Create /srv/tta/releases/ and extract code.
- Activate venv, pip install -r requirements.txt.
- Run migrations and validate.
- Switch symlink and restart systemd.
- Run smoke tests and monitor logs.

Version History

1.0	Initial production-ready SOP including CI/CD, rollback, backups, monitoring. 23 Feb 2026.
-----	---
